# Theory Assignment - 3

**Name:** Kaustubh Raj **Roll No:** 22515000065 **Section:** NC

### *1. What is the difference between frontend and backend JavaScript?*

**Frontend JS** runs in the browser. It builds the user interface, handles the DOM and events, and uses browser APIs (window, document, fetch).
**Backend JS** runs on a server using Node.js. It handles databases, files, authentication and APIs, and has no DOM or window.

### *2. What is DOM?*

The Document Object Model is a tree-like representation of an HTML page created by the browser. JavaScript uses it to access and change elements, content and styles.

```
document.getElementById("title").innerText = "Hello";
```

### *3. What is event in JavaScript?*

An event is an action that happens in the browser, such as a click, key press, form submit or mouse hover. JavaScript responds to it using event listeners.

```
btn.addEventListener("click", () => alert("Clicked"));
```

### *4. What is event loop in JavaScript?*

JavaScript is single-threaded. The event loop keeps checking whether the call stack is empty; if it is, it moves the next task from the queue (promises/microtasks first, then timers/callbacks) onto the stack. This is how async code works without blocking.

```
console.log(1);
setTimeout(() => console.log(2), 0);
console.log(3);   // Output: 1 3 2
```

### *5. What is hoisting in JavaScript?*

Hoisting means declarations are moved to the top of their scope before execution. **var** is hoisted with value undefined, function declarations are fully hoisted, and **let/const** are hoisted but cannot be used before declaration.

```
console.log(a);   // undefined
var a = 5;
```

### *6. What is event bubbling?*

When an event occurs on an element, it runs on that element first and then moves upward through its parent elements up to the document. It can be stopped using **event.stopPropagation()**. (The opposite is event capturing.)

### *7. Named vs anonymous vs arrow function*

**Named:** has a name and is hoisted. **Anonymous:** has no name, usually stored in a variable or passed as an argument. **Arrow:** shorter syntax using =>, and has no own **this**.

```
function add(a, b) { return a + b; }       // named
const add = function(a, b) { return a + b; }  // anonymous
const add = (a, b) => a + b;               // arrow
```

### *8. Explain for-of and for-in loop*

**for-of** loops over the **values** of an iterable (array, string). **for-in** loops over the **keys** (property names/indexes) of an object.

```
let arr = ["a", "b"];
for (let v of arr) console.log(v);   // a b
for (let i in arr) console.log(i);   // 0 1
```

### *9. Explain any five array methods*

**push()** - adds an element at the end. **pop()** - removes the last element. **map()** - returns a new array after applying a function to each element. **filter()** - returns elements that satisfy a condition. **reduce()** - reduces the array to a single value.

```
let a = [1, 2, 3];
a.push(4);                        // [1,2,3,4]
a.pop();                          // [1,2,3]
a.map(x => x * 2);                // [2,4,6]
a.filter(x => x > 1);             // [2,3]
a.reduce((s, x) => s + x, 0);     // 6
```

### *10. What is callback method?*

A callback is a function passed as an argument to another function and executed later, usually after a task completes. It is used in asynchronous code, event handlers and array methods.

```
function greet(name, cb) { console.log("Hi " + name); cb(); }
greet("Kaustubh", () => console.log("Done"));
```